

Fermion TeX Engine のしくみ

「書いた瞬間、印刷そのままの見た目になる」編集環境 — やさしい解説版

TeX DOM Runtime プロジェクト

2026 年 7 月 3 日

概要

このソフトは、論文や技術文書づくりの定番である LaTeX（ラテフ）を、「書くたびに数秒待って PDF を作り直す道具」から「文字を打った瞬間（約 0.03 秒）に、印刷とまったく同じ見た目が画面に出る道具」に作り替えたものです。この文書では、専門知識がない方にも分かるように、どういう仕組みで速くしたのか、そして今回の改良で「画面のプレビュー」と「本物の PDF」のズレをどうやってゼロにしたのかを説明します。細かい技術資料は別冊（docs/01～07）にあり、これはその入り口になる読み物です。

目次

1	このソフトは何をするもの？	1
2	なぜ速いのか — 「セーブポイント」方式	2
3	今回の大改良 — プレビューと本物の PDF のズレをなくした	2
3.1	何が問題だったか	2
3.2	ズレの正体 — 「別の作業台」で組んでいた	2
3.3	直し方 その 1 — 本物の紙面の流れの上で組む	3
3.4	直し方 その 2 — ページ割り付けの「規則書」を丸写しする	3
4	「本当に一致しているの？」の確かめ方	3
5	図やグラフがきれいに出なかった問題も解決	4
6	できないことも正直に	4
7	ミニ用語集	5
8	もっと詳しく知りたい人へ	5

1 このソフトは何をするもの？

LaTeX（ラテフ）は、数式や図表を含む文書をとっても美しく仕上げてくれる「組版（くみはん）ソフト」です。組版とは、原稿の文字たちを「どの行のどの位置に、どのくらいの間隔で置くか」を決めて、紙面に並べる作業のこと。新聞や書籍のような仕上がりの美しさは、この組版の計算の賢さで決まります。LaTeX はこの計算が世界一級で、だから論文の世界では 40 年以上使われ続けています。

ただし LaTeX には昔からの弱点があります。一文字直すだけでも、文書全体を最初から作り直すので、結果を見るまで数秒～数十秒待たされるのです。本を一冊まるごと刷り直さないと誤植の直りが確認できない印刷所、を想像してください。

このプロジェクトで作った Fermion TeX Engine は、その待ち時間をなくしました。

	従来の LaTeX	Fermion TeX Engine
1 か所直したとき	全体を作り直し（数秒～）	直した段落だけ組み直し（約 0.03 秒）
画面に出る見た目	PDF を開き直して確認	打った瞬間に反映
仕上がりの品質	印刷品質	同じ（本物の LaTeX が計算）

大事なのは 3 行目です。速くするために「それらしい別物」を画面に出しているのではなく、**見た目を計算しているのは本物の LaTeX (lualatex) そのものです**。行のどこで折り返すか、単語の間をどれだけ空けるか——そうした計算には一切手を加えていません。変えたのは「いつ・どの単位で計算するか」だけです。

2 なぜ速いのか — 「セーブポイント」方式

秘密はゲームで言う**セーブポイント**です。

文書を段落ごとのブロックに区切り、「ブロック 1 まで処理し終えた状態」「ブロック 2 まで処理し終えた状態」…という具合に、**組版ソフトの作業状態を丸ごと保存**していきます。どこかを直したら、文書全体をやり直すのではなく、**直した場所の直前のセーブポイントに戻って、そこから直した段落だけを組み直します**。ゲームで敵にやられたとき、最初からではなく直前のセーブから再開するのと同じ発想です。

「作業状態を丸ごと保存」と言うと重たそうですが、ここに OS（macOS や Linux）の便利な機能を使っています。**fork（フォーク）**という「動いているプログラムの完全なコピーを一瞬で作る」機能で、コピーにかかる時間は実測 0.0002 秒。LaTeX が読み込んだフォントや設定、章番号の進み具合など、何万項目もの内部状態がそっくりそのまま保存されます。

この仕組みのおかげで、1 打鍵あたりの仕事は「セーブポイントの複製＋段落 1 つ分の組版」だけになり、数ミリ秒（1 ミリ秒 = 1/1000 秒）で終わります。

3 今回の大改良 — プレビューと本物の PDF のズレをなくした

3.1 何が問題だったか

これまでの版でも見た目はほぼ正しかったのですが、本物の `lualatex` で作った PDF と見比べると、ページ内の行数が時々 1 行違う、見出しの前の空きが微妙に違う、図が壊れたアイコンになる、といったズレがありました。

プレビューは「だいたい合っている」では困ります。画面で見たものがそのまま印刷される——それがこの道具の存在意義なので、「原理的にズレが生じない構造」に作り直しました。

3.2 ズレの正体 — 「別の作業台」で組んでいた

原因をたとえ話で説明します。

これまでのエンジンは、各段落を**本の紙面とは別の、小さな作業台の上で組んでいました**。そして出来上がった段落を、あとから紙面に貼り付けていました。段落 1 つの中身はこれで完璧に組めます。ところが——

- 「前の段落との間をどれだけ空けるか」は、前の段落の最後の行の形に依存します。小さな作業台の上では前の段落が見えないので、貼り付けるときに**計算式で推測**していました。
- 「見出しの直後の段落は行頭を下げない」「見出しの直後では改ページしない」といった**前後のつながりの情報**も、作業台の外に持ち出せずに消えていました。

推測は本物と数ミクロン〜数ミリずれます。ズレはページの下に行くほど積み重なり、「あと 1 ミリで次の行が入るかどうかな」の境目では**行が丸ごと 1 本ずれる**ことになります。これが「行数が微妙に違う」の正体でした。

3.3 直し方 その 1 — 本物の紙面の流れの上で組む

v3.2 では、段落を別の作業台ではなく、**本物の紙面の流れ（`LaTeX` が実際に文書を組んでいくときの本流）の上で組み**、出来上がった結果をそのまま写し取る方式に変えました。写し取ったら本流を巻き戻して、次の編集に備えます。

こうすると、段落同士の間隔・字下げ・「ここで改ページするな」という印などが、推測ではなく**すべて `LaTeX` 自身が置いた本物**になります。実験で、この方式で写し取った中身が「普通に `lualatex` で一気に組んだ場合」と 1 バイトもたがわず**一致**することを確認しました。つまり材料のレベルで、もうズレようがありません。

3.4 直し方 その 2 — ページ割り付けの「規則書」を丸写しする

材料が本物でも、それをページに割り付ける係（どの行でページを切るか、図をページの上に置くか下に置くか、脚注の場所をどう確保するか）が我流では、やはり結果がずれます。

幸い、TeX と LaTeX には割り付けの厳密な規則書が存在します（TeX の原典プログラムと、LaTeX の出力ルーチンと呼ばれる部分です）。以前の割り付け係はこの規則書の「要約版」で動いていて、ところどころに決め打ちの数字（例：脚注の線のまわりは 6 ポイント空ける、など）が入っていました。

今回、この要約版を捨てて、規則書を一行ずつそのまま移植しました。決め打ちの数字は全廃し、必要な寸法はすべて実際に動いている LaTeX から測って受け取ります。「規則書のどこを読んでも、うちの割り付け係は同じことをする」——これが「原理的に同じ見た目になる」ことの根拠です。

4 「本当に一致しているの？」の確かめ方

主張だけでは信用できないので、自動の答え合わせ機を作りました。

同じ原稿を (a) このエンジン、(b) 本物の lualatex、の両方で処理し、すべての行の位置（何ページの、上から何ミリの位置か、行頭はどこか）を機械的に突き合わせます。人間の目視ではなく、数値での全数チェックです。

結果は次のとおりです。

検証した文書	一致した行	内容
英語デモ文書	85/85	脚注・図・表・数式・目次・参考文献入り
最小文書	3/3	
英語論文テンプレート	62/62	図表の自動配置・脚注入り
日本語テンプレート	43/43	日本語組版 (lualatexja)
数学ノートテンプレート	50/50	定理環境・複数行数式
日本語実文書	36/36	表紙・目次・改ページ・数式入り
編集を 6 回重ねた後の状態	86/86	差分更新の経路でも一致

全文書・全行が一致しました。残っている差は最大 0.02 bp。bp は印刷業界の長さの単位で、0.02 bp は約 7 ミクロン = 髪の毛の太さの 10 分の 1 以下です（これは答え合わせ機の測定粒度そのもので、事実上「ぴったり」を意味します）。

おまけの発見もありました。答え合わせに最初使っていた市販の変換ツール（PDF を画像に直すツール）が、実は座標を約 0.1% 歪めて出力することが判明したのです。偽物のズレに悩まされた末、PDF ファイルの中身を直接読む方式に切り替えて解決しました。「測定器そのものを疑う」ことも今回の大事な教訓でした。

5 図やグラフがきれいに出なかった問題も解決

図 (TikZ という作図機能) や複雑な数式は、文字と違って「本物の PDF から絵として切り出して、プレビューに貼り込む」方式にしています。本物の PDF の絵そのものなので、**原理的に印刷と同じ見た目**になります。今回、この貼り込みまわりの不具合を 4 つ直しました。

1. **図が壊れたアイコンになる**：図の画像を呼び出す名前に「#」という記号が入っており、ブラウザは「#」から先を別の意味に解釈するため、名前が途中で切れて画像を見つけれられていませんでした。名前の書き方を直して解決。
2. **図がときどき欠ける**：書き込みが終わりきっていない PDF ファイルを読んでもう瞬間がありました。「最後まで書けたか」を確認してから読むようにして解決。
3. **日本語文書で数式の位置がずれる**：日本語用の部品がページの寸法指定を上書きしてしまい、切り出し位置が狂うことがありました。切り出し方を寸法指定に頼らない方式に変えて解決。
4. **特定の設定 (hyperref) の文書で数式が重なって表示される**：この場合だけ古い方式で絵を作っていたのが原因で、新方式に揃えて解決。

現在は、図・表・数式ともに印刷そのものの見た目でプレビューされます。

6 できないことも正直に

未対応の機能もあります。ごまかしの近似はせず、「未対応」と分かる形にしています。

- 1 つの脚注が長すぎて次のページにまたがる場合の分割
- 二段組のレイアウト
- 欄外への書き込み (marginpar)

いずれも将来対応するときは、今回と同じ方針——本物の材料と本物の規則書だけを使い、答え合わせ機で全行一致を確認する——で行います。

7 ミニ用語集

LaTeX (ラテフ) / lualatex 文書組版システムの定番と、その現行エンジン。原稿 (テキストファイル) から美しい PDF を作る。

組版 (くみはん) 文字や図を紙面のどこに、どんな間隔で置くかを決める作業。

プレビュー 編集画面の横に出る「仕上がりの見た目」。本エンジンでは打った瞬間に更新され、印刷結果と一致する。

ブロック 文書を段落や見出し単位で区切ったもの。差分更新の最小単位。

セーブポイント (チェックポイント) あるブロックまで処理し終えた LaTeX の作業状態の丸ごと保存。OS の fork 機能で一瞬で作れる。

フロート 図や表を「このへんに置いて」とだけ指定し、実際の置き場所（ページ上部・下部など）を LaTeX が自動で決める仕組み。

bp (ビッグポイント) 長さの単位。1 bp $\doteq$ 0.353 mm。PDF の内部座標に使われる。

TikZ (ティクス) LaTeX の中でグラフや図形を描くための作図機能。

8 もっと詳しく知りたい人へ

エンジニア向けの完全な技術詳解（全 7 章、日本語）がリポジトリの `docs/` にあります。第 1 章が TeX の背景知識、第 3 章が本エンジンの心臓部（セーブポイント機構と今回の完全一致システム）、第 6 章が開発中に踏んだ罫の記録です。また、答え合わせ機は `tools/verify-layout.mjs` として同梱されており、任意の文書で「プレビュー＝本物の PDF」を誰でも検証できます。

この文書自体も、本エンジンの検証対象と同じ `lualatex` で組版されています。